

Ejercicio 1: integracion del catalogo externo con CheapShark

Objetivo

Analizar que parte de la API publica de CheapShark permite incorporar un catalogo externo al sistema sin guardar los juegos en nuestra base de datos.

En este contexto:

- external_game_id pasara a almacenar el gameId de CheapShark.
- La biblioteca sigue siendo propia del sistema.
- El catalogo pasa a consultarse bajo demanda en CheapShark.

Endpoint para buscar juegos por texto

El endpoint mas adecuado para buscar juegos por titulo es:

```
GET https://www.cheapshark.com/api/1.0/games?title={texto}
```

Ejemplo:

```
GET https://www.cheapshark.com/api/1.0/games?title=batman
```

Segun la documentacion publica de CheapShark, este endpoint:

- Busca juegos por title de forma case-insensitive.
- Devuelve una lista de juegos.
- Incluye para cada resultado el precio mas barato actual y su cheapestDealID.
- Permite limit con maximo 60.
- Permite exact=1 para coincidencia exacta.

Campos tipicos de respuesta:

- gameId
- steamAppID
- cheapest
- cheapestDealID
- external
- internalName
- thumb

Por que este endpoint y no /deals

CheapShark tambien permite filtrar /deals por titulo, pero ese endpoint devuelve ofertas, no juegos. Eso implica varias filas para un mismo juego, una por tienda. Para un catalogo que necesita presentar juegos y seleccionar uno para guardarlo como external_game_id, /games?title= encaja mejor.

Endpoint para consultar varios juegos por ID

El endpoint para consultar informacion de varios juegos a la vez es:

```
GET https://www.cheapshark.com/api/1.0/games?ids={id1},{id2},{id3}
```

Ejemplo:

```
GET https://www.cheapshark.com/api/1.0/games?ids=128,129,130
```

Segun la documentacion publica:

- ids es obligatorio.
- Debe enviarse como lista separada por comas.
- El maximo es 25 juegos por peticion.
- Por defecto la respuesta vuelve como objeto indexado por ID.
- Si se envia format=array, la respuesta puede devolverse como array.

Este endpoint devuelve informacion mas completa que la busqueda por texto:

- info del juego
- cheapestPriceEver
- deals asociadas al juego

Autenticacion y aspectos relevantes de la API externa

CheapShark indica que:

- La API es publica.
- No requiere autenticacion.
- No requiere API key.
- Aplica rate limiting.
- Si se supera el limite, responde con 429 Too Many Requests.
- El tiempo restante del bloqueo puede consultarse con la cabecera Retry-After.
- La API soporta CORS.
- Esta pensada para consultas en respuesta a la accion del usuario, no para reconstruir un catalogo cacheado enorme mediante scraping o descarga masiva.

Aspectos practicos a tener en cuenta en el backend:

- Conviene limitar las busquedas y no disparar peticiones innecesarias.
- Conviene agrupar lecturas por ID usando ids= cuando el frontend necesite varios juegos.
- Como external_game_id va a ser el gameId de CheapShark, el backend debe tratarlo como identificador externo estable, no como clave interna de nuestra base de datos.
- El gameId llega como valor de CheapShark y en la API aparece como identificador externo del catalogo; en nuestro sistema puede guardarse como string en external_game_id.

Relacion entre biblioteca y catalogo

La biblioteca y el catalogo no representan lo mismo:

- Biblioteca: datos propios del usuario en nuestro sistema, por ejemplo status y hours_played.
- Catalogo: informacion externa del juego obtenida bajo demanda desde CheapShark.

Por eso una entrada de biblioteca necesita guardar solo la referencia externa (external_game_id) mas el estado del usuario. El resto de informacion del juego puede resolverse despues consultando el catalogo.

Por que al frontend solo se le devuelve informacion minima del juego

Esta conclusion es una inferencia de diseno a partir de la documentacion y del enunciado.

Tiene sentido devolver solo informacion minima en la busqueda inicial por estas razones:

- La pantalla de busqueda necesita ayudar a elegir un juego, no descargar todos sus detalles y ofertas completas.
- Menos campos significa respuestas mas ligeras y menor latencia.
- Reduce el numero de datos duplicados entre frontend, backend y API externa.
- Evita acoplar demasiado el frontend al formato completo de CheapShark.
- CheapShark ya separa esta idea en su propia API: la busqueda por titulo devuelve un resumen y la consulta por ID devuelve el detalle completo.

En la practica, para una lista de resultados suele bastar con:

- gameId
- titulo (external o el nombre que el backend normalice)
- miniatura (thumb)
- precio mas barato actual (cheapest)

Si luego el usuario necesita mas detalle, el frontend puede pedirlo por ID.

Por que el catalogo no se almacena en la base de datos del sistema

Tambien es una conclusion de diseno apoyada por la documentacion publica.

No guardar el catalogo completo en nuestra base de datos tiene sentido porque:

- CheapShark ya es la fuente de verdad de esos datos.
- Los precios y ofertas cambian con frecuencia.
- Duplicar todo el catalogo generaria datos obsoletos, sincronizacion extra y mas complejidad.
- La propia documentacion recomienda usar la API en respuesta a consultas del usuario y avisa del rate limiting para evitar usos masivos.
- Nuestro sistema solo necesita persistir lo que es propio: la relacion del usuario con el juego.

En resumen:

- La biblioteca si se guarda porque es informacion propia del sistema.

- El catalogo no se guarda completo porque es informacion externa, cambiante y recuperable bajo demanda.

Decision recomendada para el proyecto

- Usar GET /api/1.0/games?title={texto} para el buscador.
- Guardar en external_game_id el gameId de CheapShark.
- Usar GET /api/1.0/games?ids={ids} cuando el frontend necesite resolver varios juegos ya referenciados en la biblioteca.
- Devolver al frontend solo un subconjunto estable y pequeno de campos.
- No persistir el catalogo completo en la base de datos local.

Fuentes

- CheapShark API endpoint: <https://www.cheapshark.com/api>
- CheapShark API docs: <https://apidocs.cheapshark.com/>
- CheapShark official Postman documentation:
<https://www.postman.com/cheapshark/workspace/cheapshark-s-public-workspace/documentation/530355-334a254b-aae7-4450-a352-b573b31403fe>